TECHNICAL ARTICLE

PostgreSQL Transaction ID Wraparound

Architecture, Risk Analysis, Monitoring, Troubleshooting, Recovery, and Prevention

Validated against PostgreSQL 18 official documentation

Prepared as an operations-focused reference | September 2026

Scope Covers normal XID and MultiXact aging, production diagnostics, emergency recovery order, safe VACUUM usage, and preventive monitoring. Commands must be tested in a lower environment and adapted to your PostgreSQL version and managed-service permissions.

Reference: https://rohithsankepally.substack.com/p/postgres-transaction-wraparound

Table of Contents

1. Executive Summary

PostgreSQL uses 32-bit transaction identifiers (XIDs) for MVCC visibility. Because normal XIDs are finite and compared using a moving “past versus future” window, old tuple XIDs must be frozen before their age becomes unsafe. VACUUM performs this freezing and advances the catalog markers relfrozenxid and datfrozenxid.

Note: Near wraparound, PostgreSQL does not become an ordinary read-only database. It warns first and, when the safety margin is almost exhausted, refuses commands that require assignment of a new XID. Read-only work that does not need a new XID may still succeed, but applications usually experience a write outage.

A healthy response is not simply “run VACUUM FREEZE everywhere.” First identify and remove blockers such as old transactions, prepared transactions, or replication slots. Then vacuum the oldest affected relations or the whole database with ordinary VACUUM, verify that XID ages fall, and correct the reason autovacuum could not keep up.

2. Core Concepts

2.1 MVCC and tuple versions

Each heap tuple stores system information that includes xmin, the inserting transaction ID, and usually xmax, the deleting or replacing transaction ID. UPDATE normally creates a new tuple version and marks the old version. Snapshot rules decide which version a transaction can see, allowing readers and writers to proceed with limited blocking.

```
CREATE TABLE wrap_demo(id integer PRIMARY KEY, status text);
INSERT INTO wrap_demo VALUES (1, 'new');
SELECT xmin, xmax, ctid, * FROM wrap_demo;

UPDATE wrap_demo SET status = 'processed' WHERE id = 1;
SELECT xmin, xmax, ctid, * FROM wrap_demo;
```

Lab note xmin and xmax are implementation-facing system columns. They are useful for learning and diagnostics, but application logic should not depend on their values remaining meaningful forever.

2.2 What VACUUM does—and does not do

- Marks space from dead tuples reusable inside the same table; standard VACUUM usually does not return that space to the operating system.
- Maintains the visibility map and freezes sufficiently old live tuple XIDs and MultiXact IDs.
- Updates relfrozenxid/relminmxid when the required pages have been scanned.
- VACUUM FULL rewrites and exclusively locks a table. It is a space-reclamation operation, not the emergency wraparound fix.

3. How Wraparound Protection Works

The normal XID space is approximately 4.29 billion values, but PostgreSQL's modular comparison rules treat about two billion IDs as being in the past and about two billion as being in the future relative to the current XID. Reserved special IDs are not normal user transaction IDs. A very old, unfrozen tuple could eventually appear to come from the future, so PostgreSQL must prevent that state.

3.1 Freezing

Freezing records that an old row version is visible to all relevant future transactions, so visibility no longer depends on the original normal XID. Modern PostgreSQL may represent the frozen state through tuple hint information rather than literally overwriting every xmin with the numeric FrozenTransactionId; the operational outcome is the same: the tuple is protected from wraparound.

3.2 Catalog age markers

Marker	Meaning	Operational use
pg_class.relfrozenxid	Oldest remaining unfrozen normal XID boundary for a relation.	Rank tables by age inside the connected database.
pg_database.datfrozenxid	Minimum relfrozenxid across relations in a database.	Rank databases and estimate cluster-wide urgency.
pg_class.relminmxid	Oldest MultiXact boundary for a relation.	Detect MultiXact wraparound risk.
pg_database.datminmxid	Database-level minimum MultiXact boundary.	Rank database MultiXact age.

4. Default Protection Thresholds

The following PostgreSQL 18 defaults are starting points, not universal tuning targets. Confirm live values with SHOW because upgrades, vendor platforms, or explicit settings may differ.

Setting	PG 18 default	Purpose
vacuum_freeze_min_age	50,000,000	Eligible XID age for freezing during VACUUM.
vacuum_freeze_table_age	150,000,000	Triggers an aggressive table scan, subject to effective limits.
autovacuum_freeze_max_age	200,000,000	Forces anti-wraparound autovacuum; server-start setting, with lower per-table override possible.

Setting	PG 18 default	Purpose
vacuum_failsafe_age	1,600,000,000	Last-resort mode: removes delays and skips some nonessential work.
autovacuum_multixact_freeze_max_age	400,000,000	Forces protection against MultiXact wraparound.

```
SHOW server_version;
SHOW autovacuum;
SHOW track_counts;
SHOW autovacuum_freeze_max_age;
SHOW vacuum_freeze_min_age;
SHOW vacuum_freeze_table_age;
SHOW vacuum_failsafe_age;
SHOW autovacuum_multixact_freeze_max_age;
```

Do not tune by folklore Lower freeze ages can cause more freezing work; higher maximum ages increase the allowed age and pg_xact/pg_commit_ts footprint. Base changes on XID consumption rate, vacuum duration, storage, and maintenance capacity.

5. Risk Signals and Monitoring

5.1 Database-level XID runway

```
SELECT datname,
       age(datfrozenxid) AS xid_age,
       current_setting('autovacuum_freeze_max_age')::bigint AS force_age,
       round(100.0 * age(datfrozenxid)
             / current_setting('autovacuum_freeze_max_age')::bigint, 2)
         AS pct_of_force_age
FROM pg_database
ORDER BY xid_age DESC;
```

Use percentage of the configured forced-vacuum age for an early operational signal, but also alert on absolute distance to the two-billion comparison boundary. A percentage above 100 does not by itself mean data loss; it means anti-wraparound work is due or active and must be investigated immediately.

5.2 Oldest tables including TOAST

```
SELECT c.oid::regclass AS table_name,
       greatest(age(c.relfrozenxid), age(t.relfrozenxid)) AS xid_age,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS total_size
FROM pg_class c
LEFT JOIN pg_class t ON c.reltoastrelid = t.oid
WHERE c.relkind IN ('r', 'm')
ORDER BY xid_age DESC
LIMIT 50;
```

5.3 MultiXact age

```
SELECT datname, mxid_age(datminmxid) AS multixact_age
FROM pg_database
ORDER BY multixact_age DESC;

SELECT c.oid::regclass AS relation,
       mxid_age(c.relminmxid) AS multixact_age
FROM pg_class c
WHERE c.relkind IN ('r','m')
ORDER BY multixact_age DESC
LIMIT 50;
```

5.4 Current vacuum progress

```
SELECT pid, datname, relid::regclass AS relation, phase,
       heap_blks_scanned, heap_blks_total,
       round(100.0 * heap_blks_scanned / NULLIF(heap_blks_total,0), 2)
         AS heap_scan_pct,
       index_vacuum_count
FROM pg_stat_progress_vacuum
ORDER BY pid;
```

6. Root-Cause Analysis

Cause	Evidence	Corrective direction
Long transaction or idle in transaction	Old xact_start and large age(backend_xmin/backend_xid) in pg_stat_activity.	Confirm owner; commit/rollback or terminate safely.
Prepared transaction	Old transactionid in pg_prepared_xacts.	Resolve with COMMIT PREPARED or ROLLBACK PREPARED after business validation.
Replication slot retention	Old xmin or catalog_xmin in pg_replication_slots.	Restore consumer or drop only a confirmed abandoned slot; replica may require rebuild.
Autovacuum capacity	Workers continuously busy; old relations wait; long vacuum duration.	Tune workers, memory/cost settings, and per-table thresholds after measurement.
Lock conflicts	Autovacuum log reports skipped relation or worker waits on lock.	Find blocker and correct DDL/maintenance scheduling.
Storage/I/O pressure	Slow scans, full filesystem, high I/O latency.	Restore headroom; throttle competing jobs; add capacity.
Per-table disabled autovacuum	reloptions contains autovacuum_enabled=false.	Re-enable; anti-wraparound workers can still start, but disabling routine maintenance is unsafe.

6.1 Long-running and idle transactions

```
SELECT pid, usename, datname, state, xact_start,
       age(backend_xid)  AS backend_xid_age,
       age(backend_xmin) AS backend_xmin_age,
       wait_event_type, wait_event,
       left(query, 200) AS query
FROM pg_stat_activity
WHERE backend_xid IS NOT NULL OR backend_xmin IS NOT NULL
ORDER BY greatest(age(backend_xid), age(backend_xmin)) DESC NULLS LAST;
```

6.2 Prepared transactions

```
SELECT gid, prepared, owner, database,
       age(transactionid) AS xid_age
FROM pg_prepared_xacts
ORDER BY xid_age DESC;
```

6.3 Replication slots

```
SELECT slot_name, slot_type, database, active,
       age(xmin) AS xmin_age,
       age(catalog_xmin) AS catalog_xmin_age,
       restart_lsn, confirmed_flush_lsn
FROM pg_replication_slots
ORDER BY greatest(age(xmin), age(catalog_xmin)) DESC NULLS LAST;
```

Safety Never terminate sessions, resolve prepared transactions, or drop replication slots based only on age. Confirm ownership, application impact, HA/DR state, and rollback/rebuild requirements first.

7. Troubleshooting Workflow

1. Record the exact warning/error, affected database, current time, PostgreSQL version, and recent changes.
2. Measure database XID and MultiXact ages; identify the oldest database.
3. Connect to that database and rank relations, including TOAST age.
4. Check whether an anti-wraparound autovacuum is already running and whether it is progressing.
5. Identify blockers: long transactions, prepared transactions, replication slots, locks, and storage pressure.
6. Remove only confirmed blockers using the safest application-aware action.
7. Run targeted ordinary VACUUM on the oldest relations, or database-wide ordinary VACUUM as a simple recovery action.
8. Recheck relfrozenxid/datfrozenxid age and logs. Repeat for every affected database, including template databases when applicable.
9. After service restoration, correct autovacuum capacity and monitoring gaps; document the incident and verify backups.

8. Emergency Recovery Procedure

Change control Use a privileged maintenance session. On self-managed PostgreSQL, a superuser is needed to ensure system catalogs are processed. Managed services may use a platform-specific administrative role and may restrict termination or settings changes.

8.1 Preserve remaining runway

- Pause nonessential writers and batch workloads through the application or connection layer.
- Do not run commands that rewrite tables or consume unnecessary XIDs.
- Keep sufficient disk headroom and monitor database and operating-system I/O.

8.2 Remove confirmed blockers

```
-- Use only after application-owner approval:
SELECT pg_terminate_backend(<pid>);

-- Resolve a verified prepared transaction:
COMMIT PREPARED '<gid>';
-- or
ROLLBACK PREPARED '<gid>';

-- Drop only a verified abandoned replication slot:
SELECT pg_drop_replication_slot('<slot_name>');
```

8.3 Vacuum the affected database

```
-- Target oldest relation first when time is critical:
VACUUM (VERBOSE, ANALYZE false) schema_name.table_name;

-- Simple database-wide recovery action, run while connected to that DB:
VACUUM (VERBOSE, ANALYZE false);
```

Do not use VACUUM FULL for wraparound recovery. In the shutdown-protection state, official guidance also recommends ordinary VACUUM rather than VACUUM FREEZE because FREEZE can do more work than the minimum required to restore XID assignment. Once normal operation is restored, a planned freezing strategy may be used where justified.

8.4 Validate recovery

```
SELECT datname, age(datfrozenxid) AS xid_age,
       mxid_age(datminmxid) AS multixact_age
FROM pg_database
ORDER BY xid_age DESC;

SELECT c.oid::regclass AS relation,
       age(c.relfrozenxid) AS xid_age,
       mxid_age(c.relminmxid) AS multixact_age
FROM pg_class c
WHERE c.relkind IN ('r','m')
ORDER BY xid_age DESC
LIMIT 50;
```

9. Prevention and Production Hardening

9.1 Monitoring policy

Control	Suggested practice
XID age	Collect datfrozenxid and top relation ages frequently; alert well before forced age.
MultiXact age	Monitor datminmxid and relminmxid separately.
Vacuum progress	Track pg_stat_progress_vacuum and unusually long phases.
Blockers	Alert on long transactions, idle-in-transaction sessions, old prepared xacts, and slot xmin/catalog_xmin age.
Logs	Set log_autovacuum_min_duration to a useful nonnegative value during tuning; retain and review skip/failure messages.
Capacity	Track worker saturation, relation growth, I/O latency, and disk free space.
Runbook	Test emergency queries, roles, escalation contacts, and managed-service restrictions.

9.2 Large-table autovacuum example

Large tables often need smaller scale factors because a percentage of a very large row count can delay vacuum too long. Values below are examples only and require workload testing.

```
ALTER TABLE app.large_orders SET (
  autovacuum_enabled = true,
  autovacuum_vacuum_scale_factor = 0.02,
  autovacuum_vacuum_threshold = 5000,
  autovacuum_analyze_scale_factor = 0.01,
  autovacuum_analyze_threshold = 5000
);

SELECT relname, reloptions
FROM pg_class
WHERE oid = 'app.large_orders'::regclass;
```

9.3 Guardrails

- Set idle_in_transaction_session_timeout where application behavior permits; understand that forced session termination rolls back its open transaction.
- Keep transactions short and avoid leaving result viewers or admin tools idle inside transactions.
- Monitor logical and physical replication consumers; define ownership for every slot.
- Avoid blanket autovacuum disabling. Use per-table tuning only with monitoring and a rollback plan.
- Plan vacuum capacity using peak XID consumption, not average traffic alone.

10. Common Mistakes

Mistake	Why it is unsafe or inaccurate
“Wraparound means PostgreSQL silently becomes read-only.”	It warns and then refuses XID-assigning commands; this is a protective shutdown behavior, not a general read-only mode.
“Dead tuples alone cause wraparound.”	Old live tuple XIDs also require freezing. Bloat cleanup and anti-wraparound protection overlap but are not identical.
“VACUUM is always single-threaded.”	Heap scanning is performed by the leader, but supported index vacuum/cleanup work can use parallel workers. Anti-wraparound work is not accurately summarized as universally single-threaded.
“Run VACUUM FULL to fix it.”	FULL rewrites and locks the table and is explicitly inappropriate in the emergency state.
“Run VACUUM FREEZE first in every emergency.”	When XID assignment is already blocked, ordinary VACUUM minimizes work; remove blockers first.
“autovacuum=off removes all protection.”	PostgreSQL can still launch anti-wraparound autovacuum, although disabling normal autovacuum remains risky.
“Raising autovacuum_freeze_max_age fixes capacity.”	It mostly postpones forced work and increases age/storage exposure; it does not remove the underlying bottleneck.

11. Operational Checklist

- ☐ XID and MultiXact ages are below defined warning thresholds.
- ☐ Oldest relations are identified in every database.
- ☐ No unexplained old backend_xmin/backend_xid exists.
- ☐ Prepared transactions have owners and aging alerts.
- ☐ Replication slots have active consumers and documented rebuild plans.
- ☐ Autovacuum workers are not continuously saturated.
- ☐ Autovacuum logs are retained and reviewed.
- ☐ Large tables have tested per-table thresholds where required.
- ☐ Emergency VACUUM permissions are verified.
- ☐ The runbook is tested in a nonproduction environment.

12. Official-Documentation Validation Notes

Article concept	Validated result
32-bit circular XID model	Correct as a mental model; comparisons use an approximately two-billion past/future horizon.
MVCC creates tuple versions	Correct, with the nuance that xmax may represent deletion, update, or locking/MultiXact states.
VACUUM removes dead tuples	Correct, but ordinary VACUUM makes space reusable internally rather than necessarily shrinking the file.
OldestXmin is simply the lowest active XID	Useful simplification only; visibility horizons also involve snapshots and retention sources.
Freezing protects old live tuples	Correct; modern physical representation should not be described only as replacing xmin with a literal value.
Near wraparound becomes read-only	Corrected: XID-assigning commands are refused after warnings, to prevent data loss.
Emergency vacuum is non-cancellable and single-threaded	Not a safe general statement; behavior depends on phase/version, and parallel index vacuum may be available. Avoid cancelling protection work without a recovery plan.

13. References

- **PostgreSQL 18 — Routine Vacuuming:** https://www.postgresql.org/docs/18/routine-vacuuming.html
- **PostgreSQL 18 — Vacuuming Configuration:** https://www.postgresql.org/docs/18/runtime-config-vacuum.html
- **PostgreSQL 18 — VACUUM Command:** https://www.postgresql.org/docs/18/sql-vacuum.html
- **PostgreSQL 18 — VACUUM Progress Reporting:** https://www.postgresql.org/docs/18/progress-reporting.html#VACUUM-PROGRESS-REPORTING
- **PostgreSQL 18 — pg_database Catalog:** https://www.postgresql.org/docs/18/catalog-pg-database.html
- **PostgreSQL 18 — Monitoring Database Activity:** https://www.postgresql.org/docs/18/monitoring.html
- **Source article — Rohith Sankepally, Postgres Transaction WrapAround:** https://rohithsankepally.substack.com/p/postgres-transaction-wraparound

Validation basis:

PostgreSQL 18 documentation was used for current behavior and defaults. Always select the documentation matching the deployed major version before applying configuration or recovery commands.